Building AI Features That Feel Fun Without Feeling Broken

Courtney Yatteau • Developer Advocate, Esri

TechBash 2026 • Thursday, October 15 • 10:20 AM

The first answer is just the start.

What happens on the next click matters just as much.

A shopping assistant

You ask for a birthday gift under $50. It suggests something clever you would not have searched for.



Then the budget changes

You switch to $30, but the older $50 suggestions arrive late and replace the new results.

A good answer still needs a dependable application.

Fun and dependable are not opposites.

FUN

A language tutor plays along as you practice ordering coffee in Spanish.

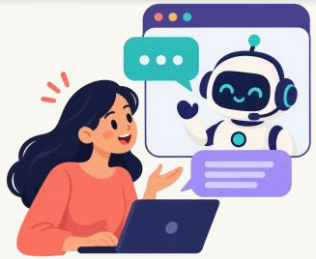


USEFUL

A shopping assistant recommends gifts that are actually in stock and within budget.

TRUSTWORTHY

A support chatbot says when it cannot confirm a delivery date instead of guessing.



The model can suggest. The app must verify.

An impressive answer is not the same as a confirmed action.

THE MODEL

An airline assistant explains flight-change options in plain language.



THE APPLICATION

Checks the actual booking, fare rules, available seats, and gets confirmation before changing anything.

AI handles flexible language. Your app owns what is real and actionable.

A quick test for every AI feature

TRUST: five habits that make the fun part usable.

T Tell users what is happening

R Retain accepted work

U Understand request ownership

S Stop and recover cleanly

T Test whether AI earns its place

TRUST is about the experience around the answer, not just the answer.

What if the next page writes itself?

Tiny Adventure · AI writes scenes, but the app owns the story rules.

Tiny Adventure

A short branching story in three scenes.



LOOK FOR Real AI writes the scene and the two choices.

THEN NOTICE The app controls the scene count and saves accepted history.

Who decides what happens next?

AI contributes

Scene title, story wording, and two possible actions.

Application controls

Three turns, valid choice IDs, accepted history, saved versions.

01-ADVENTURE.HTML / COMMITTING A SCENE

```
validateScene(payload.scene, snapshot.input.history.length + 1);
if (!canCommit(id, race)) { /* ignore stale */ return; }
```

Sometimes AI should choose a filter, not a fact.

The user says

“Something quiet and free on a rainy afternoon.”

→

The app needs

Indoor • Calm • Free

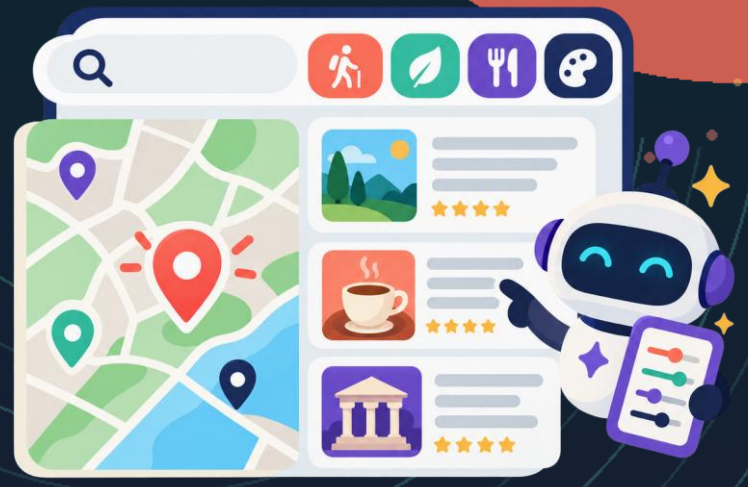
Let the model interpret intent. Let the app select known results.

Turn everyday language into useful filters.

Activity Finder · a fixed catalog, a map, and AI-suggested preferences.

Activity Finder

Natural-language preferences become bounded filters.



LOOK FOR AI suggests category, environment, and budget filters.

THEN NOTICE The app selects known activities and fixed sample map pins.

AI interprets. Your code decides.

REQUEST

“Rainy, quiet, free.”

FILTERS

Indoor • Calm • Free

KNOWN RESULTS

```
const items =
matchingActivities(filters);
```

A map can look authoritative.

That makes data provenance especially important.

Our sample catalog

Illustrative activities, fixed locations, and no live availability, weather, or distance.

Not claimed

Current hours, accurate distance, availability, or “near me.”

Polished UI must not imply facts the app never checked.

A suggestion is not a decision.

AI suggests

A friendlier headline and message body.

The user decides

Review, edit, apply, undo, or ignore it.

Nothing is silently sent or replaced.

Rewrite it. Review it. Keep control.

Text Remixer · suggestion and draft remain separate until you approve.

Text Remixer

Try a different tone without losing your original.



LOOK FOR AI proposes a friendlier or more playful rewrite.

THEN NOTICE The user edits, applies, and can undo the replacement.

Keep the suggestion separate from the draft.

SUGGESTION

Reviewable, editable, disposable.

ACCEPTED DRAFT

Changes only after Apply. Undo is explicit.

03-TEXT-REMIXER.HTML / APPROVING A REWRITE

```
undoDraft = $('draft').value;
$('draft').value = $('headline').value + '\n\n' + $('suggestion').value;
```

What does AI think you drew?

Sketch & Caption · a visual guess, editable words, your original drawing.

Sketch & Caption

A simple canvas drawing becomes a caption suggestion.

LOOK FOR The model examines the actual canvas image.

THEN NOTICE You review the description and keep the exact drawing.



A plausible caption is still a guess.

MODEL OUTPUT

“Looks like …” plus a suggested description and uncertainty.

REVIEWED OUTPUT

An editable description kept with the unchanged original sketch.

05-SKETCH-CAPTION.HTML / KEEPING THE REVIEWED RESULT

```
$('keptImage').src = canvas.toDataURL('image/png');
$('keptDescription').textContent = $('description').value.trim();
```

“AI feature” can mean four very different jobs.

CREATE · Tiny Adventure

New story content

INTERPRET · Activity Finder

Words into supported filters

TRANSFORM · Text Remixer

New wording for review

DESCRIBE · Sketch & Caption

An editable visual guess

Flexible language needs a narrow doorway.

Structured responses are the beginning of a contract, not the end.

SHAPE

Adventure expects title, text, choices, and ending. Sketch expects guess, description, and uncertainty.

RULES

Two choices in scenes one and two. Supported filters only. Short bounded text fields.

MEANING

Even well-formed output can still be a bad rewrite, a wrong caption, or a misleading filter set.

Structured output keeps the doorway narrow. Your app still decides what counts as usable.

Not all valid responses are useful.

SCHEMA PASSES

The response has the right fields and fits the app contract.

HUMAN CHECK

Does the rewrite preserve meaning? Does the caption match the drawing? Did the finder misread the request?

Validation protects the application. Review protects the user decision.

Waiting should not erase progress.

BAD FEELING

The user asks for scene two and scene one disappears while they wait.

BETTER FEELING

Keep the accepted story, draft, or sketch visible and add honest status text.

Waiting should add information, not remove usable state.

The last answer is not always the latest answer.

FIRST REQUEST

Haunted library · 4 seconds

NEWER REQUEST

Space station · 1 second

The older answer can arrive after the user has moved on.

AI is not just a response.

It is a sequence of state changes.

TOO LATE

An outdated answer replaces the latest one.

STOPPED

The user cancels while existing content should stay.

FAILED

An error happens. Now what?

Correct answer. Wrong request.

REQUEST A — Library ····· 4s

REQUEST B — Station ··· 1s

Arrival order should not decide which content the user sees.

Give every answer a reason to be accepted.

WHEN REQUEST STARTS

```
const id = ++version;
const work = new
AbortController();
```

WHEN RESPONSE ARRIVES

```
if (id !== version ||
work.signal.aborted) return;
showResult(payload, input);
```

Actual Activity Finder pattern: obsolete responses cannot update the UI.

Which answer gets to win?

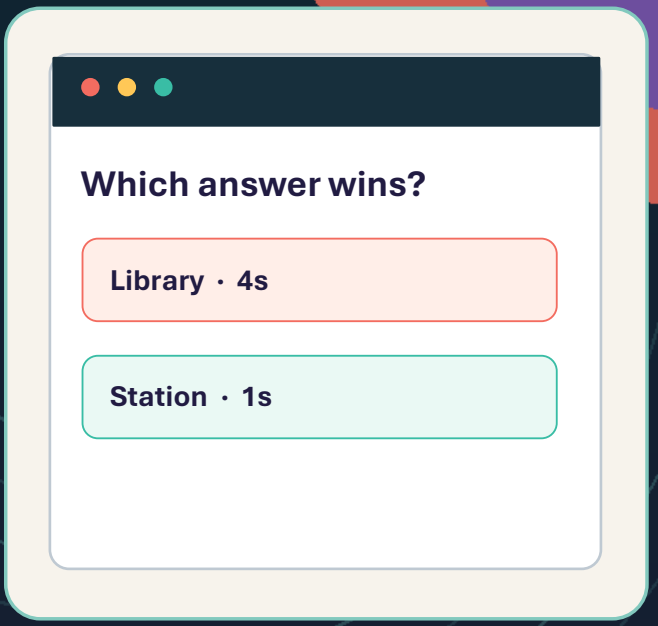
A repeatable timing experiment · no AI calls in this lab.

Late Answers

First with the ownership guard off. Then with it on.

LOOK FOR Old library response overwrites the newer station choice.

THEN NOTICE Guard on: newest choice wins; old success is ignored.



A response can finish without winning.

GUARD OFF

Older library result overwrites newer station.

GUARD ON

Older library result is ignored. Station stays.



Only the current request may commit a new result.

Stop waiting. Keep what is already good.

ACCEPTED CONTENT

Remain visible while the next answer is pending.

STOP ACTION

Cancel local waiting and prevent stale UI updates.

04-RESILIENCE-LAB.HTML / STOPWORK()

```
revision++;
operation?.abort();  operation = null;
```

Stop it. Fail safely. Keep going.

The same lab · stop a request, then recover from a bounded failure.

Stop + Recover

One accepted scene must survive both experiments.

LOOK FOR Stop a slow request without losing the current scene.

THEN NOTICE After two failures, use a clearly labeled prepared fallback.



Good failure states still move the user forward.

KEEP

Preserve what was accepted.

STOP

Don't let canceled work update the view.

RECOVER

Try again or use labeled fallback content.

Retrying forever is not a recovery strategy.

ATTEMPT 1

Temporary failure

ATTEMPT 2

One bounded retry

USER CHOICE

Try again or use fallback

A clear ceiling is better than an indefinite spinner.

We designed around real feature behavior, not just flashy outputs.

T	**Tell**	Waiting, real AI, or prepared content
R	**Retain**	Keep accepted work on screen
U	**Understand**	Only the current request updates
S	**Stop**	Cancel and recover cleanly
T	**Test**	Does AI belong here at all?

Status text is part of the product.

WAITING

“Writing scene two…”
“Interpreting your request…”

READY

“Review this suggestion before using it.”

FALLBACK

“Prepared sample, no AI call.”

Accepted work should outlive an attempt.

STILL HERE

Scene one stays visible while scene two loads. The original draft stays visible while a rewrite is pending.

NEW REQUEST

A new request starts without erasing what the person already accepted.

Attempts are temporary. Accepted state is what people care about.

Request ownership beats completion order.

IF response.requestId IS CURRENT

Accept it and update the screen.

IF response.requestId IS OLD

Ignore it, even if it succeeded, because the user has already moved on.

Every waiting state needs an exit.

STOP

Cancel local waiting and keep the current accepted state.

RETRY

Make the next attempt intentional.

FALLBACK

Use a prepared scene or sample when that is more useful than another wait.

Would a simpler feature do the job?

GOOD FIT FOR AI

Open-ended storytelling, wording changes, and sketch interpretation.

MAY NOT NEED AI

Known records, manual filters, ordinary UI, and problems with clear deterministic rules.

Use AI when it adds judgment or generation. Do not use it just to sound modern.

When should you actually use AI?

1 · Is the request open-ended, ambiguous, or genuinely easier in natural language?

2 · Can the person still review, correct, or reverse the result?

3 · Can your app handle late answers, failure, and fallback honestly?

Let the model be flexible. Keep the contract small.

INPUT

Valid request, current state, and any accepted history

MODEL

Suggested story, rewrite, filter interpretation, or caption

APP

Validate, review, commit, label, and recover

That boundary is what lets the feature feel fun without feeling broken.

Three improvements you can try in one feature.

LABEL IT

Show pending, real AI, prepared, and error states.

GUARD IT

Reject responses from old requests.

KEEP IT

Preserve accepted work through stops, retries, and fallbacks.

The model can surprise people with the answer.

The interface should not surprise them about the process.

Delight needs trust.

Keep the person, not the model, in control.

Build the fun part. Own the dependable part.

Thanks, TechBash!

Courtney Yatteau

@c_yatteau

courtneyyatteau

@cyatteau.bsky.social

Slides / demos / resources